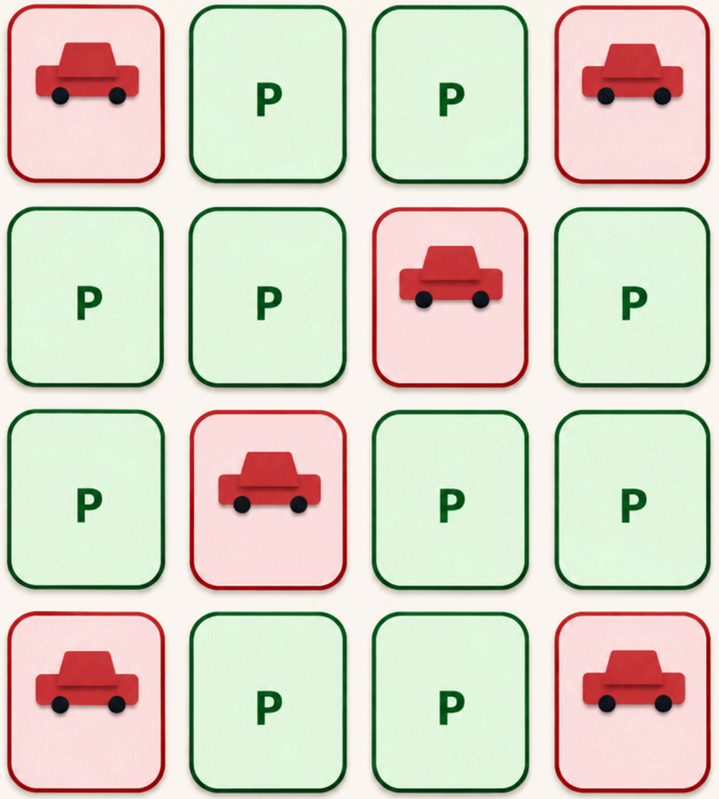
SmartPark AI

Smart Parking Space Detection



TEAM MEMBERS

Manik Dev Azad 20233178
Prakhar Patni 2023545
Roshna Rajesh 20233551
Rudra Pratap Singh 20233552

INTRODUCTION

Project Overview



Problem

Finding available parking spaces can be time-consuming. Manual monitoring makes it difficult to identify vacant spaces and understand parking occupancy efficiently.



Solution

We developed SmartPark AI, a computer vision system that analyzes parking-lot images using YOLO to classify spaces as vacant, occupied, or unknown.



Why It Matters

By visualizing parking availability and occupancy statistics, the system helps users understand parking conditions and supports more informed parking decisions.

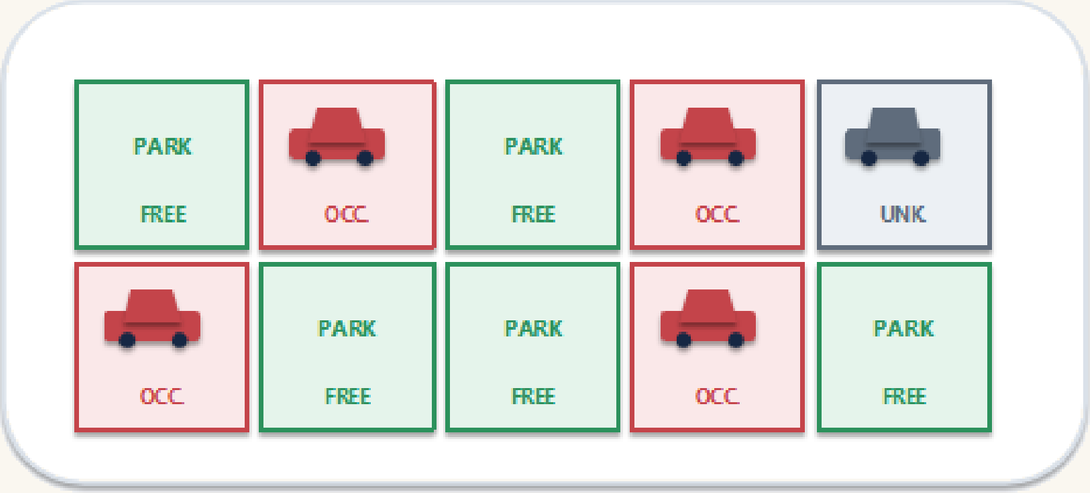
Project Objective

Main Goal

Develop an AI-powered parking detection system that identifies parking availability from parking-lot images.

Supporting Objectives

1. Detect Parking Spaces: Use YOLO-based computer vision to identify parking spaces and analyze their status.
2. Classify Occupancy: Categorize spaces as vacant, occupied, or unknown and calculate occupancy statistics.
3. Visualize Results: Build an interactive dashboard that displays annotated images, parking availability, and occupancy metrics.



What We Found

1

Upload image
User selects a parking-lot image

2

Validate & decode
Backend checks and reads image bytes

3

YOLO inference
Detects parking bays and predicts labels

4

Compute metrics
Counts vacant, occupied and unknown bays

5

Visualize results
Overlay + dashboard + per-bay table

Key Insight

Input: one parking image → Output: annotated image + structured bay predictions + summary statistics

System Architecture

Frontend

React 19 + Vite

File upload & preview
Analysis state via useAnalysis
KPI cards and results table
Annotated image display

Backend

FastAPI + Pydantic

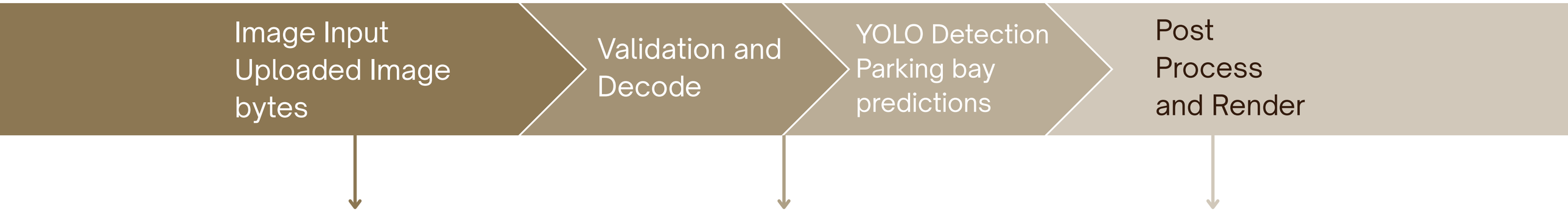
POST /api/analyze
Image validation & decoding
Structured response schemas

ML / IMAGE LAYER

Ultralytics YOLO + OpenCV

Thread-safe model loading
Parking-bay inference
Bounding-box annotations
Base64-encoded result image

Detection Pipeline



Configured inference settings

Confidence threshold: 0.25
Image size: 1280 (configurable)

Per-bay prediction fields
ID • model label • status • confidence • bounding-box coordinates

Output is an annotated image and structured predictions

The detector uses the configured Hugging Face model repository: simahanyan/parking-lot-yolo26m.

Occupancy Classification and Metrics

VACANT
Green bounding box

OCCUPIED
Red bounding box

UNKNOWN
Grey bounding box

Each detection includes a confidence score and coordinates; the UI can inspect individual bay results.

Occupancy percentage

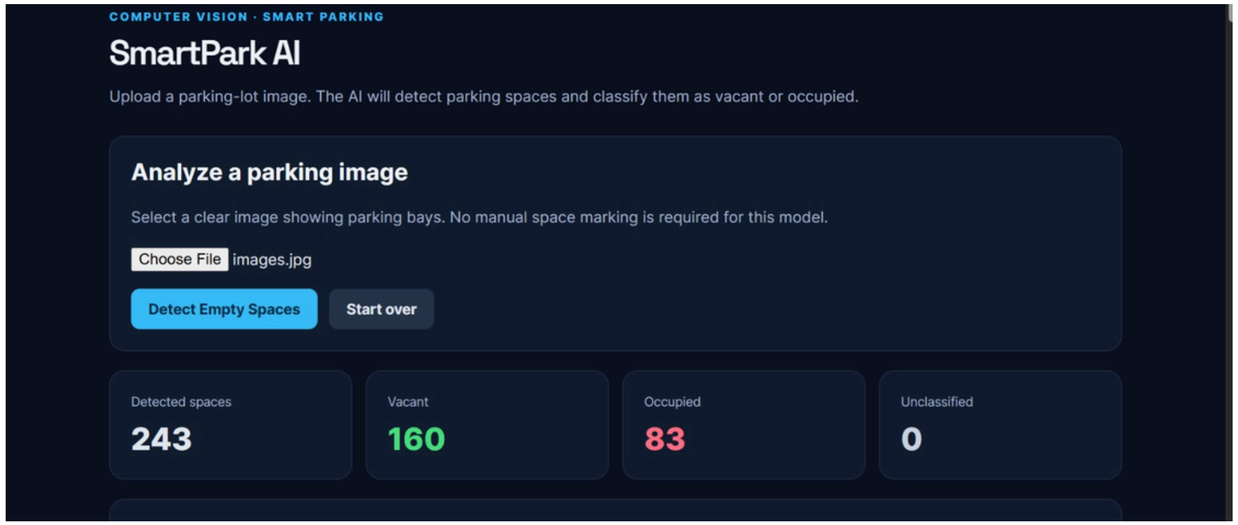
Occupied ÷ (Occupied + Vacant) × 100

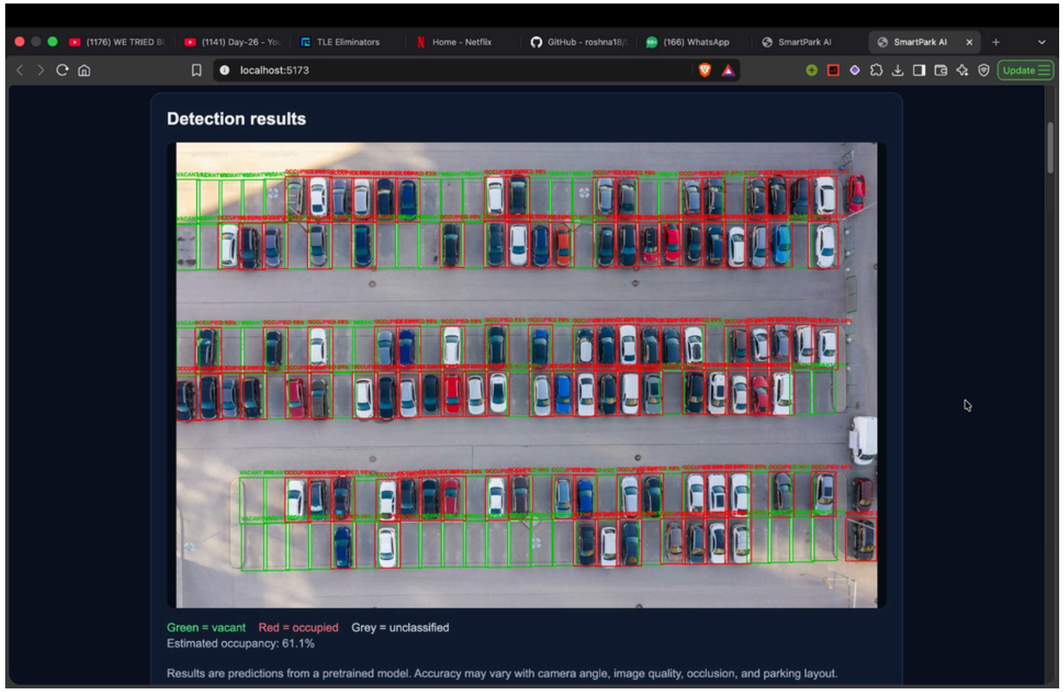
Unknown predictions are excluded from the denominator.

Summary Returned

Occupied ÷ (Occupied + Vacant) × 100

Total detected bays
Vacant / occupied / unknown counts
Occupancy percentage and availability flag





Application Interface & Outputs

What the user receives

Upload and preview an image
Annotated image with colored boxes
Occupancy KPI cards
Per-bay table with confidence
Error and loading states

Kindly note: Designed for image-based analysis — not continuous live video.

TECHNOLOGY STACK, API & DATA FLOW

Technology Stack

Frontend

React 19 · Vite · JavaScript/JSX · CSS

Backend API

Python 3.10+ · FastAPI · Uvicorn · Pydantic

Computer Vision

OpenCV · NumPy · Image decoding and annotation

Detection Model

Ultralytics YOLO · Configurable model weights

Validation & Testing

Pytest · Integration tests · Vitest

Run / Deployment

Local development · Docker Compose

API & Data Flow

Image Analysis — POST /api/analyze

- Upload and validate image; apply rate limits.
- Return occupancy metrics, predictions, confidence scores, and an annotated image.

Space Layout — /api/spaces

- Retrieve and save parking-space layouts.

Storage & Reliability

- Local JSON storage with atomic writes.
- Pydantic validation and centralized error handling.

Reliability, Testing & Limitations

Reliability Measures

- Validate image type, size (under 15 MB), and content.
- Pydantic request/response validation.
- Rate limiting for analysis requests.
- Thread-safe model loading and inference.
- Atomic JSON writes and structured error handling.

Testing & Known Limitations

- Backend unit and integration tests using Pytest.
- Frontend component tests using Vitest.
- Predictions depend on image quality, camera angle, occlusion, and parking layout.

Limitation: The current system analyzes uploaded images. Overall accuracy has not yet been verified on a representative test dataset.